

Probability and Random Walk: Exercise Guide for 1A

Maryam Ahsan

Mathematical Methods for Physicists II

Python Notebook Guide

This is a collection of notes compiled from my Mathematical Methods for Physicists II coursework on coin tossing, probability and random walks. It accompanies the exercises in 1A, including extensions on ensemble spreading and confinement, and explains how to translate their mathematical models into computational steps. The shared foundations guide should be read first.

1 Exercise 1: Running coin counts and fractions

1.1 Task and mathematical target

Simulate fair coins for 100 and 600 tosses. Plot running head and tail counts and the running head fraction. After n tosses,

$$H_n + T_n = n, \quad f_n = H_n/n, \quad E[f_n] = 1/2, \quad \text{Std}(f_n) = 1/(2\sqrt{n}).$$

1.2 Develop the algorithm

Use two current counters and lists for their histories. For each experiment:

1. Reset counters and histories.
2. Generate one of two equally likely outcomes.
3. Increase the appropriate counter.
4. Store both totals and the number of completed trials.
5. Store the head count divided by that trial number.

Python indexes from zero, so `step + 1` is the completed trial number. The two experiments use separate sequences of draws; the 100-toss result is not automatically the first part of the 600-toss result.

1.3 Checks and interpretation

Check $H_n + T_n = n$ and $0 \leq f_n \leq 1$. The fraction should fluctuate around one half. Larger samples generally have smaller fractional fluctuations, even though absolute count differences can grow.

2 Exercise 2: A one-dimensional trajectory

2.1 Task and mathematical target

Generate a fair unit-step walk from zero and plot position against step count:

$$X_{n+1} = X_n + \xi_{n+1}, \quad \xi_{n+1} \in \{-1, +1\}.$$

2.2 Develop the algorithm

Create a function accepting a step count and returning a position history:

1. Set the current position to zero and store it.
2. Choose left or right equally.
3. Update the position and append it.
4. Repeat until all steps are completed; return the history.

The history contains signed positions, not accumulated distance. The final list entry is the endpoint.

2.3 Checks and interpretation

For N steps, the history has length $N + 1$, starts at zero, and has consecutive differences ± 1 . Its endpoint has the same parity as N . Zero theoretical mean applies across realizations, not to each endpoint.

3 Exercise 3: Time occupancy and cumulative occupancy

3.1 Task and mathematical target

Count positions visited after each step, normalize the counts, and form the cumulative distribution:

$$\hat{\mu}(x) = \frac{\text{visits to } x}{N}, \quad \hat{F}(x) = \sum_{y \leq x} \hat{\mu}(y).$$

Time occupancy answers where this path spent its recorded time. Its CDF gives the fraction of observations at or below a threshold.

3.2 Develop the algorithm

1. Exclude the initial entry from the history.
2. Count occurrences using a dictionary or **Counter**.
3. Construct integer positions from the observed minimum through maximum.
4. Process positions in increasing order and divide counts by N .
5. Maintain a running sum of the normalized counts.
6. Plot occupancy bars and a cumulative step function.

The initial entry is excluded because there are N post-step observations. Include the largest position when constructing a range; Python's range excludes its upper limit.

3.3 Checks and interpretation

Verify $\sum_x \hat{\mu}(x) = 1$ and $\hat{F}(x_{\max}) = 1$ within rounding. The CDF never decreases. These correlated, varying-age positions are not independent endpoints at fixed age. An unrestricted infinite-line fair walk has no normalizable stationary position law.

4 Exercise 4: An ensemble of walks

4.1 Task and mathematical target

Generate 3000 walks of 400 steps. Measure the endpoint histogram, mean position and mean squared displacement:

$$E[X_n] = 0, \quad E[X_n^2] = n, \quad \sigma_{400} = 20.$$

Because the origin is zero, position equals displacement from the starting point.

4.2 Develop the algorithm

First generate paths: repeat the walk function, retain each complete history, and collect its endpoint. Then analyse paths: at each step index, collect positions across walkers, average them, and separately average their squares. The data can be viewed as a table with rows for walkers and columns for step counts. Column zero contains initial positions. Average across rows at fixed column. Do not confuse

$$\frac{1}{M} \sum_m X_m^2 \quad \text{with} \quad \left(\frac{1}{M} \sum_m X_m \right)^2.$$

The first is a second moment; the second is the squared mean.

4.3 Histogram logic

After 400 unit steps, only even endpoints are accessible. Use width-two bins centred on reachable sites. Density normalization gives unit total bar area; individual bar areas estimate bin probabilities.

4.4 Checks and interpretation

The mean fluctuates near zero and the second moment follows n . The standard error of the mean is $\sqrt{n/M}$: about 0.365 at $n = 400$, $M = 3000$. A small nonzero mean is expected. The approximate Gaussian endpoint shape and linear second-moment growth check different aspects of the model.

5 Exercise 5: A two-dimensional trajectory

5.1 Task and mathematical target

Choose four cardinal directions equally. One coordinate changes per tick. For step length a ,

$$E[X_N^2] = E[Y_N^2] = Na^2/2, \quad E[R_N^2] = Na^2.$$

5.2 Develop the algorithm

1. Initialize both coordinates and their histories at zero.
2. Draw one of four equally likely integers and map it to a direction.
3. Update the corresponding coordinate only.
4. Store both coordinates after every step.
5. Plot paired coordinates in the plane with equal axis scaling.

5.3 Checks and interpretation

For unit steps, $|\Delta x| + |\Delta y| = 1$, and both histories contain $N + 1$ entries. The plot is one realization; $E[R_N^2] = N$ is an ensemble statement. Moving both coordinates independently every tick would define a different model.

6 Exercise 6: Confinement to a square

6.1 Task and boundary rule

Keep the walker in $-10 \leq x, y \leq 10$. Reject proposed outward moves and count them. A rejected move leaves the walker stationary but advances time.

6.2 Develop the algorithm

Separate proposal from acceptance:

1. Copy current coordinates into proposed coordinates.
2. Apply the random increment to the proposal.
3. Test whether either proposed coordinate exceeds the bounds.
4. Reject an invalid proposal without moving; otherwise accept it.
5. Record the actual position after every proposal.

Testing before overwriting the actual coordinates makes rejection straightforward. Equality with an edge is allowed; only proposals strictly outside are rejected.

6.3 Checks and interpretation

All stored coordinates must stay in the square. Each step is a unit cardinal move or a stationary rejection. Divide rejections by all proposed steps to obtain the rejection fraction. There are $21^2 = 441$ sites. Uniform site probabilities are stationary for this rule, although finite paths need not sample equilibrium evenly. This rejecting boundary differs from the absorbing edges of the diffusion grid solver.